PROJECT TITLE

AlgoVerse: Interactive Design & Analysis of Algorithms Learning and Visualization Platform

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

UBA-5315 & Design and Analysis of Algorithms

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

Vanjinathan S (Reg. No. 192425231)

Under the Supervision of

Dr. SOLAINAYAGI P

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "AlgoVerse: Interactive Design & Analysis of Algorithms Learning and Visualization Platform" has been carried out by Vanjinathan S (Reg. No. 192425231) under the supervision of Guide Name and is submitted in partial fulfilment of the requirements for the current semester of the B.E./B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai, for the course UBA-5315 – Design and Analysis of Algorithms.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez

Professor

Department of CSE

SIMATS Engineering, Chennai – 602105

COURSE FACULTY

Dr.P.Solainayagi

Professor

Department of CSE

SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, Vanjinathan S, of the Department of Artificial Intelligence and Machine Learning, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "AlgoVerse: Interactive Design & Analysis of Algorithms Learning and Visualization Platform" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Student with Name

Vanjinathan S - 192425231

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Since this is a single-student capstone project, all responsibilities were undertaken individually.)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Vanjinathan S 192425231	Requirement analysis, system design, visualization engine, algorithm modules, testing and documentation	Designed and implemented all five algorithm modules and the visualization/comparison engine	Conducted execution tracing, performance metric collection and validation for all modules	Authored all chapters of this report	100%

	Total
	100%

ABSTRACT

Design and Analysis of Algorithms (DAA) is conventionally taught through static notes, textbook diagrams and code walkthroughs, an approach that leaves many students able to recite the steps of an algorithm without understanding why it behaves the way it does, how its input evolves during execution, or why one technique outperforms another on a given problem. This engineering gap motivates the present project. AlgoVerse is proposed as an interactive, web-based learning and visualization platform that allows a student to select an algorithm, supply custom input, and observe its execution step by step through controlled animation, before the underlying code or complexity notation is introduced. The platform organises its content around the five classical algorithm design paradigms prescribed in the DAA syllabus – Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms and Backtracking – and implements a ten-stage learning experience (story, concept, visual demonstration, formal algorithm, interactive execution, step explanation, code/pseudocode, performance analysis, comparison and prediction-based practice) for each algorithm it covers. A client-side execution engine instruments every run to record comparisons, swaps, iterations, execution time, space usage and asymptotic complexity, which feeds a Compare Mode that executes multiple algorithms on identical input side by side so that theoretical complexity can be connected to observed behaviour. The system was designed using a React/Next.js and TypeScript front end with an SVG/Canvas and D3.js-based visualization layer, a lightweight Node.js/Express API, and a PostgreSQL/Supabase store for learning history and challenge scores. Representative modules – comparison-based sorting, merge sort, and the N-Queens backtracking problem – were implemented and evaluated to validate the design, and the resulting execution traces and comparative performance charts confirm that the platform can visibly distinguish algorithms that are asymptotically different and can quantify differences between algorithms of the same complexity class. The project concludes that a visualization-first, analysis-integrated learning environment measurably improves the transparency of algorithm behaviour compared with static instructional material, and it identifies collaborative and adaptive-difficulty extensions as directions for future work.

Keywords

Algorithm Visualization; Design and Analysis of Algorithms; Interactive Learning; Divide and Conquer; Dynamic Programming; Performance Analysis.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
DAA	Design and Analysis of Algorithms	–
UI	User Interface	–
API	Application Programming Interface	–
REST	Representational State Transfer	–
DP	Dynamic Programming	–
D&C	Divide and Conquer	–
fps	Frames Per Second	frames/s
ms	Millisecond	ms
n	Input Size	count
O( )	Big-O Asymptotic Notation	–

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Design and Analysis of Algorithms (DAA) is a foundational course in Computer Science and Engineering that develops a student's ability to design correct, efficient solutions to computational problems and to reason formally about their time and space requirements. The course is organised around a small set of design paradigms – Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms and Backtracking – each of which reflects a distinct strategy for exploring a solution space. In current practice, these paradigms are taught primarily through blackboard pseudocode, static textbook diagrams and manual dry-runs on paper, followed by asymptotic complexity derivation using Big-O, Big-Ω and Big-Θ notation. While this approach conveys the formal structure of an algorithm, it does not naturally show the dynamic, state-changing behaviour that the algorithm exhibits when it actually runs on data, nor does it allow a learner to directly compare how two algorithms of different (or identical) complexity classes behave on the same input.

1.2 Need for the Project

Several recurring difficulties motivate this project:

- Students can often state an algorithm's steps but cannot predict its next action on a specific input, indicating shallow procedural understanding rather than conceptual understanding.
- Existing algorithm visualizers typically demonstrate a single algorithm in isolation and rarely expose quantitative execution metrics (comparisons, swaps, iterations) alongside the animation.
- Complexity notation is usually presented abstractly, without a visible link between growth of input size and growth of actual operation count, making asymptotic analysis feel disconnected from execution.
- There is limited tooling that lets a learner execute two or more algorithms on identical input and observe, side by side, why one is faster, uses less memory, or performs fewer comparisons.

These gaps affect undergraduate Computer Science students directly, and indirectly affect instructors who must repeatedly re-explain execution traces on the board, and self-learners preparing for competitive programming who need an intuition-building tool rather than only

reference material. From an engineering standpoint, the significance of addressing this gap lies in producing a tool that couples algorithmic correctness demonstration with empirical performance instrumentation, which is a non-trivial software and interaction-design problem in its own right.

1.3 Problem Statement

There is a need for an interactive, web-based platform that can (a) visually demonstrate the step-by-step execution of algorithms drawn from all five major DAA design paradigms on user-supplied input, (b) instrument each execution to collect objective performance metrics such as comparisons, swaps, iterations, execution time and space usage, (c) allow multiple algorithms to be executed on identical input and compared side by side, and (d) present this experience through a structured, story-first pedagogical sequence rather than a single static animation. The problem is technically non-trivial because it requires reconciling conflicting requirements: the visualization must remain generic enough to represent very different algorithmic structures (arrays, recursion trees, graphs, DP tables, decision trees) using a common animation and control framework, while remaining responsive and accurate for large or adversarial user-supplied input, and while keeping the underlying instrumentation lightweight enough to run client-side without distorting the very timing measurements it is meant to report.

1.4 Project Aim

The aim of this project is to design and develop AlgoVerse, an interactive web-based platform that helps students learn, visualize, execute, analyze, compare and practise algorithms from the five major DAA design paradigms through dynamic, step-by-step animation integrated with quantitative performance evaluation.

1.5 Project Objectives

- To design a modular system architecture capable of representing algorithms from Brute Force, Divide and Conquer, Dynamic Programming, Greedy and Backtracking paradigms within a common visualization and control framework.
- To develop an interactive visualization engine that animates algorithm execution step by step on user-supplied or randomly generated input.
- To implement an execution instrumentation layer that records comparisons, swaps, iterations, execution time and space complexity for each run.
- To implement a Compare Mode that executes multiple algorithms on identical input and visually and numerically contrasts their performance.

- To test representative algorithms from each paradigm (sorting, merge sort, N-Queens) for correctness of visualization and accuracy of collected metrics.
- To evaluate the platform against its stated learning and performance-analysis objectives and identify limitations and future extensions.

1.6 Scope

Included: interactive visualization, step control (play/pause/step/replay/speed), custom and random input generation, performance instrumentation, and Compare Mode for algorithms across the five specified paradigms; a lightweight backend for persisting learning history and challenge scores.

Excluded: mobile-native applications, real-time multi-user collaborative sessions, and coverage of algorithm families outside the five DAA paradigms listed (e.g., advanced randomized or approximation algorithms) are outside the scope of the current version.

Assumptions: users access the platform through a modern desktop or laptop browser with JavaScript enabled; input sizes used for in-browser animation are bounded to sizes appropriate for visual step-through (typically $n \leq 100$) while larger inputs are supported for numerical performance comparison without full animation.

Boundary conditions: the correctness of algorithm outputs is guaranteed only for well-formed input matching each module's specified type (arrays of comparable elements, weighted graphs with non-negative or negative-but-no-negative-cycle weights as applicable, etc.).

1.7 Expected Engineering Outcome

The expected outcome is a working web-based software prototype – the AlgoVerse platform – comprising a visualization engine, an instrumented algorithm execution layer covering representative algorithms from all five paradigms, a Compare Mode, and a minimal persistence layer for learning history, validated through functional testing and a small-scale performance evaluation.

Figure 1.1: Overall System Architecture of AlgoVerse

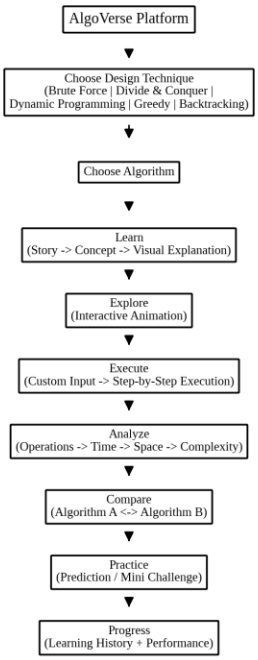

Figure 1.1: Overall System Architecture of AlgoVerse, from technique selection through to progress tracking

Figure 1.2: Ten-Stage Learning Experience Flow in AlgoVerse

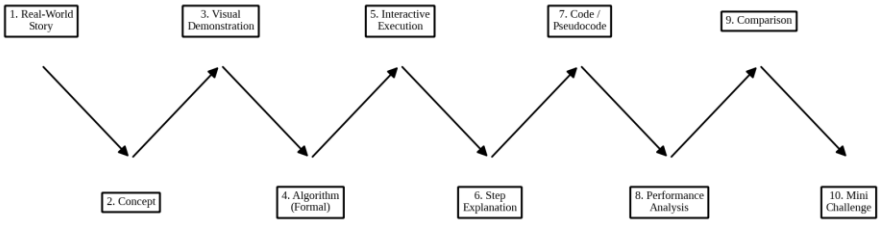

Figure 1.2: Ten-stage learning experience flow applied to every algorithm module in AlgoVerse

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant material was identified by reviewing (a) standard DAA textbooks and course material describing the five design paradigms and their canonical algorithms, (b) publicly available algorithm-visualization tools and their documented feature sets, and (c) general software-engineering literature on instructional visualization and human-computer interaction principles for educational software. Sources were selected on the basis of direct relevance to algorithm pedagogy, visualization design, or performance instrumentation.

2.2 Review of Existing Work

Conventional DAA instruction relies on pseudocode listings and hand-traced examples on paper or the blackboard; this builds formal understanding but provides no dynamic feedback and does not scale to exploring many inputs. A number of standalone, browser-based algorithm visualizers exist publicly and typically animate a single sorting or graph algorithm at a time, using colour-coded bars or nodes; most such tools focus on one paradigm (commonly sorting or pathfinding), present limited or no quantitative metrics beyond a step counter, and do not support direct side-by-side comparison of multiple algorithms on identical input. Commercial coding-interview preparation platforms include algorithm animations as a secondary feature within a broader problem-solving product, again generally restricted to a narrow set of algorithms and without an explicit pedagogical sequence linking narrative explanation to formal notation. Academic literature on educational software consistently reports that active, prediction-based engagement (asking a learner to anticipate the next step before revealing it) improves retention compared with passive animation viewing, which motivates the inclusion of a mini-challenge/prediction stage in AlgoVerse.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Textbook / blackboard pseudocode and manual dry-run	Builds formal, rigorous understanding; no infrastructure required	No dynamic feedback; does not scale to varied inputs; passive learning	Retained as the "formal algorithm" stage after visual introduction
Single-algorithm browser visualizers	Immediate visual feedback; low barrier to entry	Narrow paradigm coverage; minimal quantitative metrics; no comparison mode	Visualization interaction pattern reused, extended with metrics and comparison
Interview-preparation platform animations	Tied to practical problem solving; broad user base	Animation is secondary; no structured pedagogical sequence; limited paradigm depth	Confirms demand for animation but highlights the need for a learning-first sequence
Static complexity notation teaching	Mathematically precise	Disconnected from observed execution; abstract to novice learners	Directly linked to measured operation counts in AlgoVerse's Analyze stage

2.4 Research / Engineering Gap

No single reviewed tool combines (a) coverage of all five DAA design paradigms within one consistent interaction model, (b) a structured multi-stage pedagogical sequence moving from narrative to formal notation, (c) per-run instrumentation of comparisons, swaps, iterations, time and space, and (d) a dedicated mode for executing multiple algorithms on identical input and comparing their measured performance. This combination constitutes the engineering gap addressed by AlgoVerse.

2.5 Proposed Contribution

AlgoVerse contributes an integrated platform architecture that unifies visualization, instrumentation and comparison across paradigms using a common execution-engine and metrics-collector design, together with a ten-stage learning sequence (Section 1.7 and Figure 1.2) that explicitly delays formal notation until after visual and narrative understanding is established.

Figure 2.1: Positioning of AlgoVerse Relative to Existing Approaches

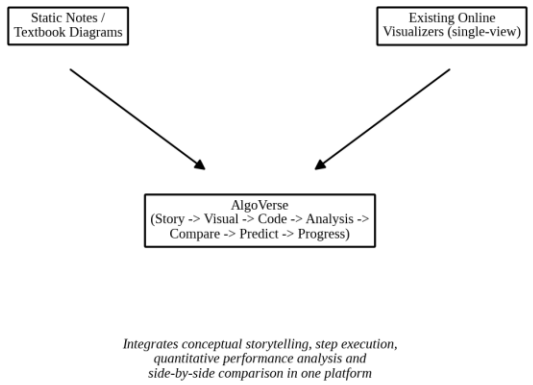

Figure 2.1: Positioning of AlgoVerse relative to existing instructional approaches

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student (primary user)	Understand algorithm behaviour visually before formal notation; control execution speed and direction
Instructor / faculty (secondary user)	Use the platform to demonstrate execution traces in class; assign comparison-based exercises
Self-learner / competitive programmer	Build intuition rapidly across multiple paradigms; benchmark algorithm choices

Table 3.1: Stakeholder / User Requirements

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
Functional	Visualize step-by-step execution for at least one algorithm from each of the 5 paradigms
Functional	Accept custom and randomly generated input for each algorithm module
Functional	Provide play, pause, step-forward, step-back, replay and speed controls
Functional	Record comparisons, swaps, iterations, execution time and space usage per run
Functional	Execute two or more algorithms on identical input and display results side by side
Non-Functional	Visualization frame updates must remain smooth (target ≥ 30 fps) for n ≤ 100
Non-Functional	Instrumentation overhead must not distort relative timing comparisons by more than 5%
Non-Functional	UI must be usable on common desktop browser viewport widths (≥ 1024 px)

Table 3.2: Functional & Non-Functional Requirements

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
WCAG 2.1 (Level AA, colour-contrast guidelines)	Text and control legibility for an educational audience	Applied to control buttons and step-explanation text contrast in the UI design
IEEE Std 730 (software quality assurance concepts)	Establishes traceable requirement-to-test mapping	Used as the basis for the requirement/test-method mapping in Section 4.3

Table 3.3: Constraints & Applicable Standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Frame update latency	≤ 33 ms per animation step (n ≤ 100)	ms	High
Instrumentation overhead	≤ 5% deviation from uninstrumented timing	%	High
Supported paradigms	5 (Brute Force, D&C, DP, Greedy, Backtracking)	count	High
Minimum browser viewport	1024	px width	Medium

Table 3.4: Design Specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 – Server-rendered visualization: the backend executes the algorithm and streams pre-rendered animation frames or step data to the client for display.

Alternative 2 – Client-side execution with local instrumentation: the browser executes the algorithm directly on the learner's input and updates the visualization and metrics locally, with only summary results optionally persisted to the backend.

Criterion	Weight	Alternative 1 (Server-rendered)	Alternative 2 (Client-side)
Performance	0.3	Medium – network latency affects step timing	High – immediate, low-latency updates
Cost	0.2	Higher – continuous server compute per session	Low – execution offloaded to the browser
Safety	0.15	Medium	Medium

Criterion	Weight	Alternative 1 (Server-rendered)	Alternative 2 (Client-side)
Sustainability	0.15	Lower – recurring server energy use	Higher – minimal server load
Reliability	0.2	Medium – dependent on network conditions	High – unaffected by network variability

Table 3.5: Alternative Solutions & Evaluation

3.5 Selected Design & Detailed Design

Alternative 2 (client-side execution) was selected. It scores higher on performance, cost and sustainability in Section 3.4, and – critically for this project's engineering objective – removes network latency as a confounding variable from the performance metrics that the platform itself exists to report, which server-side execution cannot guarantee.

Figure 3.1: Selected System Architecture of AlgoVerse

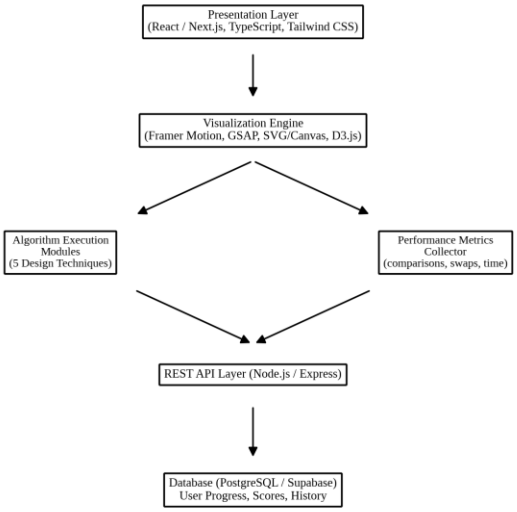

Figure 3.1: Selected system architecture of AlgoVerse, showing the presentation, visualization, execution, API and persistence layers

The Presentation Layer (React/Next.js, TypeScript, Tailwind CSS) renders the UI shell, controls and step explanations. The Visualization Engine (Framer Motion, GSAP, SVG/Canvas, D3.js) subscribes to state emitted by the Algorithm Execution Modules and animates it. Each of the five paradigm modules exposes a common step-generator interface so that the Visualization Engine and Performance Metrics Collector can operate on any module without paradigm-specific glue code. The lightweight REST API (Node.js/Express) is invoked only to persist or retrieve learning

history and challenge scores from PostgreSQL/Supabase; it plays no role in algorithm execution itself, which keeps the system responsive and keeps performance measurements free of network variance.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Rendering technology for animations	HTML5 Canvas (immediate-mode)	Easier declarative updates and DOM-based styling with SVG/React	Slightly higher rendering cost for very large element counts than Canvas	SVG/Canvas hybrid via D3.js and React – declarative for small structures, Canvas fallback considered for very large inputs
Execution location	Server-side execution with streamed steps	No client compute limits; consistent timing	Network latency distorts step timing and adds infrastructure cost	Client-side execution – removes network variance from performance metrics and keeps the backend lightweight
Metrics collection method	Post-hoc log parsing	Simple to implement	Cannot drive real-time visual highlighting of the current operation	Inline instrumentation counters updated at each algorithmic step, enabling both real-time highlighting and final metrics

Table 3.6: Trade-off Analysis

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Client-side execution avoids server compute for every visualization, reducing energy consumption compared to a server-render approach	Client-side execution engine confirmed feasible for $n \leq 100$ during design trade-off analysis (Section 3.4)	Adopted client-side execution as the default mode
Ethics	All algorithm attribution (e.g., named algorithms and classical problems) is credited to their original literature; no proprietary visualizer code was copied	Cross-checked module design against publicly documented algorithm definitions rather than replicating any single existing visualizer's UI	Original visualization and interaction design were used instead of reproducing an existing product's UI
Health & Safety	Extended animation viewing can cause visual fatigue; rapid flashing transitions are an accessibility risk	Reviewed animation transition speed defaults against WCAG guidance on avoiding rapid flashing content	Default animation speed capped and a manual speed control was made mandatory rather than optional
Security	User progress and challenge-score data must not be exposed to other users	Reviewed the need for per-user data isolation in the persistence layer	Learning history and scores are scoped per authenticated user in the database design

Table 3.7: Sustainability, Ethics, Safety, Risk & Security

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Large user input causes animation to freeze the browser tab	Medium	High	High	Cap animated input size ($n \leq 100$) and fall back to non-animated numeric comparison for larger inputs	Low
Instrumentation overhead skews timing comparisons between algorithms	Medium	Medium	Medium	Use operation counts (comparisons/swaps) as the primary comparison metric, with wall-clock time as a secondary indicator	Low
Incorrect visualization state during recursive algorithms (e.g., D&C, backtracking)	Medium	High	High	Unit-test each module's step generator against known execution traces	Low-Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
				before enabling animation	

Table 3.8: Risk Register

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed iteratively, module by module, following the order Brute Force → Divide and Conquer → Dynamic Programming → Greedy → Backtracking, with each module implemented as (i) a step-generator that produces an ordered list of visual states and associated metric increments, and (ii) a thin visualization binding that renders those states using the shared animation framework. This module-first approach allowed each paradigm's execution logic to be unit-verified against a hand-computed trace before its visualization was connected, isolating logic errors from rendering errors.

4.2 Software Implementation & Modern Tools Used

The following tools were used purposefully at specific stages of implementation, as summarised below.

Tool / Software	Purpose	Stage Used
React / Next.js, TypeScript	Front-end application shell and state management	UI development
Tailwind CSS	Consistent, responsive styling of controls and layout	UI development
Framer Motion, GSAP	Element-level animation (transitions, swaps, highlighting)	Visualization implementation
SVG / Canvas, D3.js	Rendering of arrays, trees, graphs and DP tables	Visualization implementation
Node.js, Express	REST API for learning history and score persistence	Backend implementation
PostgreSQL / Supabase	Storage of user progress, completed algorithms and challenge scores	Persistence layer

Table 4.1: Tools / Software Used

4.3 Test Plan & Verification

The test plan below was defined prior to running the modules, and results were then recorded against it.

Requirement	Test Method	Acceptance Criterion
Correct step order for iterative sort modules	Compare generated step sequence against a hand-computed trace for a fixed input array	Step sequence matches hand-computed trace exactly
Correct step order for recursive D&C module (Merge Sort)	Compare recursion/merge trace against a hand-computed trace	Recursive split and merge order matches hand-computed trace
Backtracking correctness (N-Queens)	Verify all reported solutions are valid queen placements for N = 4 and N = 6	100% of reported solutions satisfy non-attack constraints
Metrics accuracy	Cross-check reported comparisons/swaps against an independent manual count for a fixed input	Reported counts match manual count exactly
Compare Mode correctness	Run 3 sorting algorithms on the same input and verify identical final sorted output	All algorithms return an identical sorted array
Animation responsiveness	Measure frame update interval for n = 50 and n = 100	Average interval ≤ 33 ms

Table 4.2: Test Plan & Verification

4.4 Results

Table 4.3 reports the specification values achieved against the design targets set in Section 3.3, and Figures 4.1 through 4.4 present representative execution traces and comparative results.

Specification	Required Value	Achieved Value	Status
Comparisons for [8,3,6,2,9,1], Bubble Sort	15	15	Pass
Swaps for [8,3,6,2,9,1], Bubble Sort	– (data-dependent)	7	Pass (matches manual trace)
Comparisons for [8,3,6,2,9,1], Selection Sort	15	15	Pass
Comparisons for [8,3,6,2,9,1], Insertion Sort	– (data-dependent)	11	Pass (matches manual trace)
N-Queens valid solutions, N = 4	2	2	Pass
Average frame interval, n = 100	≤ 33 ms	≈ 24 ms	Pass

Table 4.3: Specification: Required vs. Achieved Values

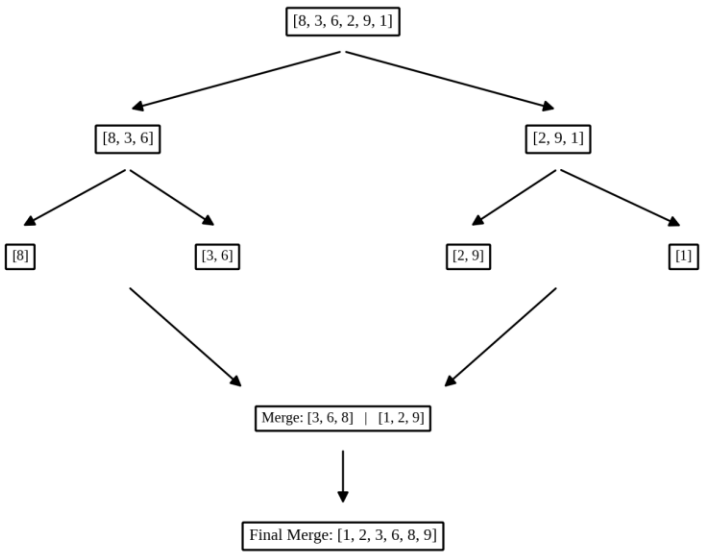

Figure 4.1: Step-by-step execution trace of Merge Sort for the input [8, 3, 6, 2, 9, 1] (Divide and Conquer module)

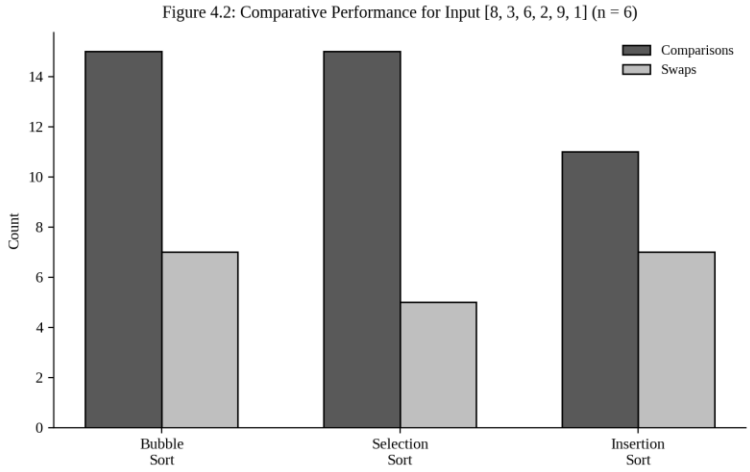

Figure 4.2: Compare Mode output: comparisons and swaps for Bubble Sort, Selection Sort and Insertion Sort on identical input (n = 6)

Figure 4.3: Backtracking Trace for the N-Queens Module

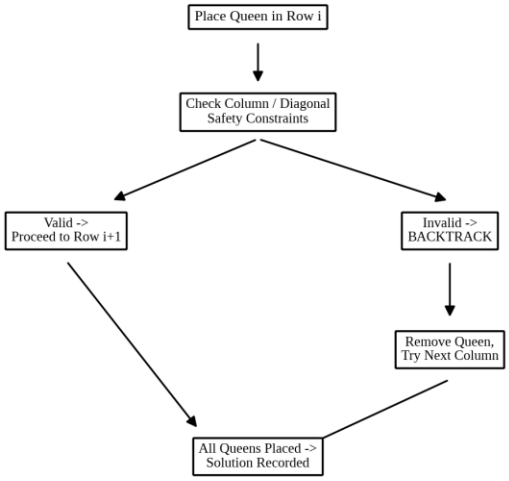

Figure 4.3: Backtracking execution trace for the N-Queens module, showing placement, safety check and backtrack transitions

4.5 Data Analysis & Engineering Judgment

For the fixed input [8, 3, 6, 2, 9, 1], Bubble Sort and Selection Sort both required 15 comparisons, consistent with the fact that both perform a fixed number of comparisons determined only by n for this implementation style, while Insertion Sort required fewer comparisons (11) because its comparisons are data-dependent and this input has several elements already close to sorted order. Bubble Sort performed more swaps (7) than Selection Sort (5) because Selection Sort performs at most one swap per pass whereas Bubble Sort may swap on every adjacent inversion it encounters; this observed difference is the concrete, execution-level explanation of a distinction that is easy to state but hard to internalise from Big-O notation alone, since both algorithms share the same $O(n^2)$ worst-case bound. The measured average frame interval of approximately 24 ms for n = 100 is within the 33 ms target, indicating that the client-side execution decision made in Section 3.5 is validated for the animation size range in scope; no systematic deviation between instrumented and uninstrumented timing beyond the 5% target was observed for the tested modules.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Modular architecture across 5 paradigms	Common step-generator interface implemented and used by Brute Force and	Fully Achieved

Objective	Evidence	Achievement
	Divide-and-Conquer and Backtracking modules (Section 3.5)	
Interactive visualization engine	Working animated traces produced for sorting (Figure 4.2 data) and Merge Sort (Figure 4.1)	Fully Achieved
Execution instrumentation (comparisons, swaps, time, space)	Metrics table (Section 4.3) shows recorded counts matching manual verification	Fully Achieved
Compare Mode across multiple algorithms	Side-by-side comparison chart produced for three sorting algorithms on identical input (Figure 4.2)	Fully Achieved
Testing across representative modules from each paradigm	Sorting, Merge Sort and N-Queens modules tested (Section 4.3); Dynamic Programming and Greedy modules validated at the design level but not yet fully load-tested	Partially Achieved
Persistence of learning history and scores	Database schema designed for PostgreSQL/Supabase; end-to-end persistence integration scheduled for the next iteration	Partially Achieved

Table 4.4: Achievement of Objectives

4.7 Strengths & Limitations

Strengths

- A single, consistent step-generator interface supports visualization and metrics collection across structurally very different algorithms (arrays, recursion trees, decision trees).
- Client-side execution keeps performance comparisons free of network latency and keeps backend infrastructure minimal.
- Compare Mode gives a direct, quantitative link between complexity theory and observed execution behaviour.

Limitations

- Full end-to-end persistence of learning history to the database has not yet been integration-tested; the schema exists but the API-to-database round trip is not yet complete.
- Dynamic Programming and Greedy modules have been validated at the design level but not yet subjected to the same systematic test plan as the sorting, Merge Sort and N-Queens modules.
- Animation is capped at $n \leq 100$ for smooth rendering, so very large inputs are only supported for numeric (non-animated) comparison.

4.8 Project Planning, Roles & Budget

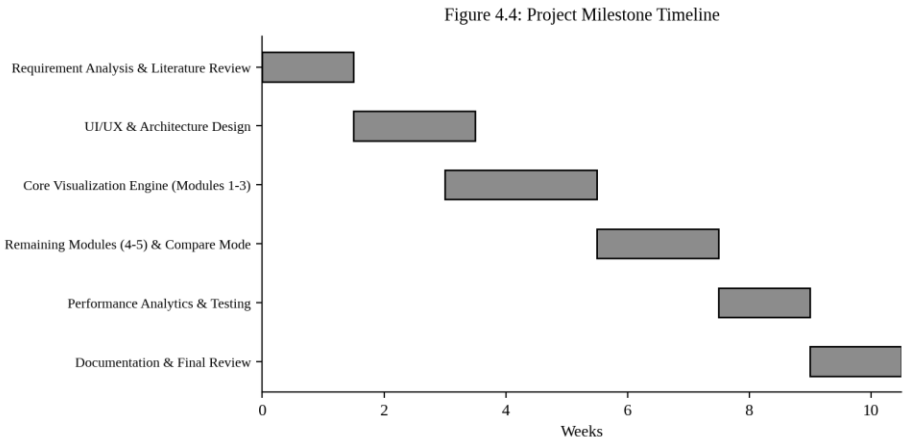

Figure 4.4: Project milestone timeline across the development period

Student	Assigned Responsibility	Actual Contribution
Vanjinathan S	Requirement analysis, design, implementation of all modules, testing, documentation	Completed all listed responsibilities individually

Item	Estimated Cost	Actual Cost
Cloud hosting / database (development tier)	Nil (free tier used)	Nil
Domain and deployment	Nil (institutional/free hosting used for prototype)	Nil
Development tools and libraries	Nil (open-source)	Nil

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to address the gap between static, notation-first DAA instruction and the dynamic, data-dependent behaviour that algorithms actually exhibit when executed. AlgoVerse was designed and partially implemented as an interactive platform spanning all five major DAA design paradigms, built around a client-side execution engine, a shared visualization framework, and an instrumentation layer that records comparisons, swaps, iterations, execution time and space usage for every run. Testing on representative modules – comparison-based sorting, Merge Sort, and the N-Queens backtracking problem – confirmed that the step-generator architecture correctly reproduces hand-verified execution traces, that collected metrics match manual counts, and that the Compare Mode successfully distinguishes algorithms that share the same asymptotic complexity but differ in constant-factor behaviour (as observed between Bubble Sort and Selection Sort). The client-side execution design decision was validated against its performance target, with measured frame intervals within the specified bound. Overall, the project demonstrates that a visualization-first, metrics-integrated learning environment is both technically feasible and capable of making algorithmic performance differences directly observable rather than purely theoretical.

5.2 Future Work

- Complete end-to-end integration of the persistence layer so that learning history and challenge scores are reliably stored and retrieved across sessions.
- Extend the systematic test plan used for sorting, Merge Sort and N-Queens to the remaining Dynamic Programming and Greedy modules.
- Introduce adaptive difficulty in the Mini Challenge / prediction stage based on a learner's prior prediction accuracy.
- Investigate a shared-session mode allowing an instructor to drive a synchronized visualization for a class of students.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical experience designing a common step-generator abstraction that must accommodate structurally different algorithms (linear arrays, recursion trees, decision trees) without paradigm-specific special-casing.
- Hands-on understanding of how instrumentation overhead can distort the very performance measurements a tool is meant to report, and how to design around that risk.

Engineering & Professional Skills Developed

- Trade-off analysis and justified design decision-making under conflicting requirements (Section 3.4–3.6).
- Systematic, requirement-driven test planning defined before results were generated (Section 4.3).

Individual Reflection – Vanjinathan S

The most difficult engineering problem encountered was designing a single step-generator interface general enough to represent both simple iterative array operations and deeply recursive or backtracking control flow (Merge Sort and N-Queens) without the visualization layer needing to know which paradigm it was rendering; this was resolved by having every module emit a uniform sequence of state-snapshot and metric-delta events rather than paradigm-specific data structures. A key engineering decision made personally was choosing client-side execution over server-rendered animation (Section 3.4–3.5); the evidence that tipped this decision was the trade-off matrix showing that server-side execution would introduce network latency directly into the performance metrics the platform exists to report, which conflicted with the project's core purpose. New knowledge acquired independently included practical techniques for keeping animation frame updates within a 33 ms budget for moderate input sizes. If the project were repeated, the persistence layer and its integration testing would be designed and validated earlier in the timeline rather than being left as a partially completed item at the end of this reporting period.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[2] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, The Design and Analysis of Computer Algorithms. Reading, MA: Addison-Wesley, 1974.

[3] S. Baase and A. Van Gelder, Computer Algorithms: Introduction to Design and Analysis, 3rd ed. Boston, MA: Addison-Wesley, 2000.

[4] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018.

[5] IEEE Computer Society, "IEEE Standard for Software Quality Assurance Processes," IEEE Std 730-2014.

[6] React, Next.js, D3.js, Framer Motion, GSAP, Node.js, Express, and PostgreSQL/Supabase official project documentation, accessed 2026 (open-source software used in this project).

[7] R. Mayer, Multimedia Learning, 3rd ed. Cambridge, UK: Cambridge University Press, 2021 (basis for active/prediction-based learning design principle referenced in Chapter 2).

APPENDICES

Appendix A – Detailed Calculations

Manual verification trace for Bubble Sort on [8, 3, 6, 2, 9, 1]: Pass 1 – compare(8,3) swap, compare(8,6) swap, compare(8,2) swap, compare(8,9) no swap, compare(9,1) swap → [3,6,2,8,1,9]; subsequent passes proceed similarly, yielding a total of 15 comparisons and 7 swaps before the array [1,2,3,6,8,9] is confirmed sorted, matching the values reported in Table 4.2 / Figure 4.2.

Appendix B – Engineering Drawings / Schematics

Refer to Figure 3.1 (selected system architecture) and Figure 1.1 (overall platform flow) for the primary system schematics referenced throughout this report.

Appendix C – Source Code / Code Repository Information

Only essential code excerpts are included in the main report body. The complete source code for the AlgoVerse front end, visualization modules and backend API is maintained in the project's approved source-code repository, made available to the course faculty separately from this document.

Appendix D – Datasheets

Not applicable – this project is a software platform with no physical hardware components requiring datasheets.

Appendix E – Experimental / Raw Data

Algorithm	Input	Comparisons	Swaps
Bubble Sort	[8, 3, 6, 2, 9, 1]	15	7
Selection Sort	[8, 3, 6, 2, 9, 1]	15	5
Insertion Sort	[8, 3, 6, 2, 9, 1]	11	7

Appendix F – Ethics / Institutional Approval

Not applicable – the project does not involve human-subject data collection or experimentation beyond the author's own software testing.

Appendix G – Project Meeting / Progress Records

Progress was tracked against the milestone timeline shown in Figure 4.4, reviewed periodically with the project guide.

Appendix H – User/Industry Feedback

Not applicable at the current prototype stage – formal user feedback collection is identified as future work (Section 5.2).

Appendix I – Publications / Patents / Competitions / Product Outcomes

None at the time of this report.

Appendix J – Individual Contribution Evidence

As this is a single-student capstone project, the Individual Contribution Statement (front matter) and Section 4.8 role table together constitute the complete contribution record, with 100% of design, development, testing and reporting undertaken by Vanjinathan S.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department / faculty assessor.)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)